

République Algérienne Démocratique et Populaire

Ministère de la Défense Nationale

École Militaire Polytechnique



Filière : Génie Informatique

Implémentation et Évaluation des Performances d'un Réseau SDN

Encadré par :

Cdt. LEBBAZ Nouredine

Cne. KHAMES Ishak

Réalisé par :

EOC. HIDOUS Karim

EOC. AOUTTI Wasim

Année Universitaire

2025/2026

Remerciements

Tout d'abord, nous tenons à exprimer toute notre gratitude et nos remerciements au **Col. ACHOURI Ali**, Directeur du Centre de Recherche-Développement des Transmissions, qui a mis à notre disposition tous les moyens humains et matériels afin de nous permettre de passer la période du stage dans de bonnes conditions. Nos sincères remerciements vont également au **Col. JWINI Abd El Hafid**, Chef du Département de Systèmes d'Information et Génie Logiciel. Nous exprimons aussi toute notre reconnaissance à nos encadreurs, le **Cdt.LEBBAZ Noureddine** et le **Cne. KHAMES Ishak**, qui ont dirigé ce travail et ont assuré le suivi quotidien avec patience, disponibilité et écoute, contribuant largement à l'aboutissement de ce rapport. Leurs réflexions et leurs conseils judicieux ont été d'une valeur inestimable du point de vue scientifique. Nous leur sommes également reconnaissants pour le temps conséquent qu'ils nous ont accordé ; nous avons beaucoup appris à leurs côtés.

Table des matières

Liste des figures	IV
Liste des abréviations	V
Introduction générale	V
I Présentation de l'établissement d'accueil	2
I.1 Introduction	3
I.2 Missions et organisation du CRDT	3
I.3 Composition du CRDT	3
I.4 Présentation du Département Systèmes d'Information et Génie Logiciel . .	4
I.5 Conclusion	4
II PRÉSENTATION DU RÉSEAU SDN	5
II.1 Introduction	6
II.2 Software-Defined Networking (SDN)	6
II.2.1 Définition	6
II.2.2 Composants principaux (Contrôleur, Plan de données, Plan de contrôle)	6
II.2.3 Interfaces de communications	8
II.2.4 Protocole OpenFlow	9
II.3 Avantages et caractéristiques des réseaux définis par logiciel (SDN)	11
II.4 Cas d'utilisations du SDN	12
II.5 Conclusion	14
III Implémentation du Réseau SDN	15
III.1 Introduction	16
III.2 Emulateur et contrôleur SDN	16
III.2.1 Emulateur Mininet	17

III.2.2 Contrôleur Open Networking Operating System (ONOS)	17
III.2.3 Docker	20
III.2.4 Open vSwitch	20
III.3 Outils d'évaluation	21
III.4 Outils de Connexion et de Gestion à Distance	22
III.4.1 PuTTY	22
III.4.2 Xming Server	22
III.5 Configuration de l'environnement	23
III.5.1 Démarrage d'ONOS :	23
III.5.2 Test de la connectivité vers la vm	24
III.5.3 Etablissement de connection ssh entre la machine hôte et la machine virtuelle	25
III.5.4 Validation de la configuration de l'environnement SDN via une simulation de topologie	26
III.6 Conclusion	27
IV Évaluation des Performances du Réseau SDN	28
IV.1 Introduction	29
IV.2 Méthodologie	29
IV.2.1 Topologies des réseaux étudiés	29
IV.2.2 Métriques de comparaison	30
IV.2.3 Scénarios de simulation et collecte des données	30
IV.2.4 Résultats et Analyse	32
IV.3 Conclusion	34
Conclusion générale	35
Bibliographie	37
Annexes	41

Liste des figures

I.1	Organigramme du CRDT/1ère RM	4
II.1	Architecture réseau défini par logiciel [6].	7
II.2	Architecture Openflow [16]	9
II.3	openflow protocole [16]	10
III.1	plateforme de simulation [25].	16
III.2	interface graphique MiniEdit [26].	17
III.3	Caractéristiques d'ONOS [27]	18
III.4	Interface Graphique Utilisateur (GUI) d'ONOS [27]	19
III.5	Interopérabilité d'ONOS avec Docker dans un Environnement SDN [30]. .	20
III.6	Commutateur virtuel Open vSwitch utilisé dans la simulation SDN [31]. .	21
III.7	Test de connectivité entre la machine virtuel et le conteneur ONOS	24
III.8	Résultat de la commande ping entre la machine hôte et la machine virtuelle.	24
III.9	Configuration de la connexion SSH entre la machine hôte et la machine virtuelle.	25
III.10	Configuration de PuTTY avec redirection X11 pour Xming.	25
III.11	Interface CLI de la machine virtuelle via une connexion SSH sécurisée. . .	26
III.12	Exécution d'un script Python pour créer une topologie réseau SDN dans Mininet.	26
III.13	Topologie réseau détectée et affichée dans l'interface graphique d'ONOS. .	27
IV.1	Variation du débit en fonction du nombre de hôtes	33
IV.2	Délai en fonction du nombre des commutateurs OVS pour les différents scénarios	34

Liste des abréviations

API	Application Programming Interface
ASIC	application-specific integrated circuit
CLI	Command-Line Interface
CRDT	Centre de Recherche et Développement des Transmission
GUI	Graphical User Interface
IP	Internet Protocol
IPFIX	Internet Protocol Flow Information Export
LACP	Link Aggregation Control Protocol
MTU	Maximum Transmission Unit
NetFlow	Network Flow
NBI	Northbound interface
ONOS	Open Network Operating System
OVS	Open vSwitch
REST	Representational State Transfer
RSPAN	Remote Switched Port Analyzer
SDN	software-defined networking
SDWN	Software Defined Wireless Network
SSH	Secure Shell
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
VLAN	Virtual Local Area Network
VM	Virtual Machine
VSDN	Virtual Software-Defined Networking

INTRODUCTION GÉNÉRALE

INTRODUCTION GÉNÉRALE

Ce rapport présente les travaux réalisés au cours du stage, qui se sont concentrés sur l'étude et l'évaluation des réseaux définis par logiciel (SDN). Cette technologie révolutionnaire, qui repose sur la séparation du plan de contrôle et du plan de données, offre une gestion centralisée et programmable des réseaux. Face à l'augmentation exponentielle des volumes de trafic, due à la prolifération des terminaux mobiles, à l'essor des réseaux sociaux, et aux tendances comme l'Internet des objets (IoT), les infrastructures réseaux traditionnelles atteignent rapidement leurs limites en termes de flexibilité, de scalabilité et d'efficacité. Le SDN s'impose ainsi comme une alternative flexible et innovante pour répondre à ces nouveaux besoins.

Dans ce contexte, le rôle du SDN est à la fois technique et stratégique. Sur le plan technique, il permet une automatisation avancée des réseaux grâce à des protocoles comme OpenFlow, simplifiant la gestion et la maintenance tout en réduisant les coûts d'exploitation. Sur le plan stratégique, le SDN prépare les infrastructures réseautiques à faire face à des défis tels que la cybersécurité, l'optimisation des ressources et l'évolution rapide des besoins en termes de qualité de service (QoS).

Le rapport est structuré en quatre chapitres principaux. Dans le premier chapitre, nous présentons l'établissement d'accueil, en décrivant son organisation, ses missions, ainsi que le département dans lequel le stage a été réalisé. Le deuxième chapitre introduit les concepts fondamentaux des réseaux définis par logiciel, en explorant leur architecture, leurs avantages et leur impact sur la gestion des réseaux modernes.

Dans le troisième chapitre, nous présentons la méthodologie adoptée et l'environnement utilisé pour la mise en œuvre et l'évaluation des SDN. Cela inclut la configuration des outils, l'utilisation des plateformes de simulation comme Mininet et ONOS, et l'adoption de technologies telles que Docker pour garantir un environnement flexible et reproductible. Enfin, le quatrième chapitre est dédié à l'analyse des résultats obtenus, mettant en lumière les avantages et les limites des SDN dans divers scénarios pratiques.

Ce rapport vise à fournir une vision claire des apports du SDN en termes de flexibilité, de performance et d'optimisation des ressources, tout en soulignant son rôle central dans le développement des réseaux modernes.

Chapitre I

PRESENTATION DE L'ETABLISSEMENT D'ACCUEIL

I.1 Introduction

Dans ce chapitre, nous présentons l'organisation et les missions du Centre de Recherche et Développement des Transmissions (CRDT/1°RM). Nous expliquons aussi comment sont organisés les différents départements de recherche et développement, en particulier le Département des Systèmes d'information et Génie Logiciel, où nous avons réalisé notre stage.

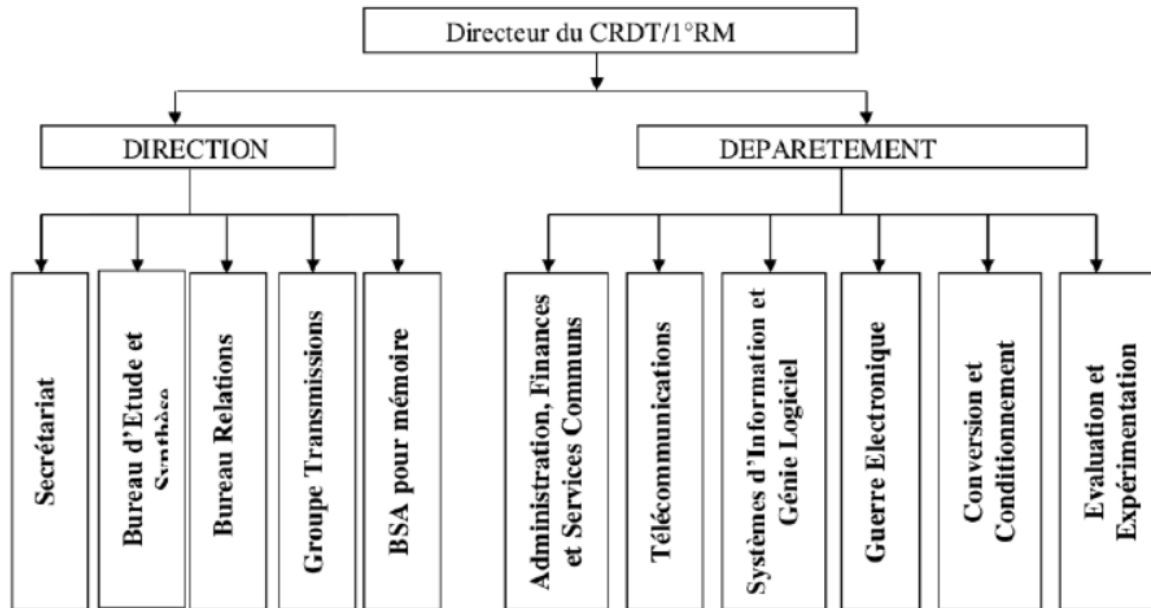
I.2 Missions et organisation du CRDT

À ce titre, il est chargé de :

- Assurer la maîtrise d'œuvre de programmes et projets de recherches scientifiques et de développement technologiques dans le domaine en relation avec la transmission
- Participer à l'évaluation, à l'expertise et à l'homologation des produits, matériels, équipements et système de communication, en vue de leur admission en service opérationnel.
- Assurer une veille scientifique et technologique en rapport avec son objet et de réunir les éléments nécessaires à l'identification et à la proposition de nouveaux axes de recherche scientifique et de développement technologique ;
- Participer aux activités de normalisation et d'assurance-qualité dans ses domaines de transmissions, de la guerre électronique, de la sécurité de communication, de l'identification Ami-Ennemi et des Systèmes de Commandement.
- De favoriser l'acquisition, la maîtrise et la diffusion des connaissances scientifiques, techniques et technologiques et de contribuer à la modernisation des équipements de transmission
- De recueillir et de traiter les informations scientifiques et techniques liées à son objet et d'en assurer la conservation et la diffusion conformément à la réglementation en vigueur au sein du Ministère de la Défense Nationale.

I.3 Composition du CRDT

Le Centre de Recherche et Développement des Transmissions est organisé selon l'organigramme illustré dans la figure I.1 :

**FIGURE I.1**Organigramme du CRDT/1^{ère} RM

I.4 Présentation du Département Systèmes d'Information et Génie Logiciel

Notre stage est effectué au niveau du Département Systèmes d'Information et Génie Logiciel, ce dernier est composé d'un :

- Laboratoire des Systèmes d'Information Géographique et des Algorithmes Décisionnels
- Laboratoire des Réseaux Informatiques et des Applications Spécifiques
- Laboratoire des Technologies de l'Information et de la Communication

I.5 Conclusion

Dans ce chapitre, nous avons présenté un bref aperçu sur la structure, l'organisation et les missions de l'établissement d'accueil. De même, nous avons présenté le département Systèmes d'information et génie logiciel dont laquelle nous avons exécuté notre stage pratique. Dans le chapitre suivant, nous allons présenter les concepts théoriques en relation avec notre stage pratique.

Chapitre II

PRÉSENTATION DU RÉSEAU SDN

II.1 Introduction

L'émergence des technologies de l'information et de la communication, telles que le cloud computing, le big data, l'internet des objets (IoT) et la virtualisation des serveurs réseau, a entraîné une explosion des volumes de trafic, alimentée par la multiplication des terminaux mobiles, l'usage intensif des réseaux sociaux et des tendances technologiques génératrices de grandes quantités de données. Face à ces évolutions, les réseaux traditionnels atteignent rapidement leurs limites en termes de flexibilité, de scalabilité et d'efficacité [1]. Pour relever ces défis, le réseau défini par logiciel (SDN) s'impose comme une solution innovante, offrant une gestion centralisée et programmable des réseaux grâce à la dissociation du plan de contrôle et du plan de données. Cette architecture permet une flexibilité accrue et une agilité optimale, permettant ainsi aux infrastructures réseau de répondre efficacement aux besoins croissants en trafic et en applications modernes.

II.2 Software-Defined Networking (SDN)

II.2.1 Définition

Le réseau défini par logiciel (SDN) représente un modèle réseau innovant, conçu pour offrir une flexibilité et une agilité accrues dans la gestion des infrastructures. Ce modèle repose sur la dissociation du plan de contrôle, responsable de la gestion des politiques et de la prise de décisions stratégiques, et du plan de données, qui assure le transfert du trafic au sein du réseau [2]. En séparant ces deux fonctions, le SDN permet une gestion centralisée et entièrement programmable des réseaux, orchestrée par un contrôleur externe, connu sous le nom de « Contrôleur SDN ». Ce contrôleur communique avec les équipements du plan de données, tels que les commutateurs et routeurs, en utilisant des protocoles de communication, notamment OpenFlow [3]. Grâce à OpenFlow, le contrôleur peut définir et déployer des règles de traitement du trafic sur les équipements réseau, offrant ainsi une gestion dynamique, évolutive et réactive des réseaux [4]. Cette approche simplifie non seulement la configuration et la maintenance des réseaux, mais permet également une adaptation rapide aux besoins changeants des applications modernes et des environnements de trafic en constante évolution.

II.2.2 Composants principaux (Contrôleur, Plan de données, Plan de contrôle)

Le réseau défini par logiciel (SDN) repose sur une architecture composée de trois composants principaux : le contrôleur, le plan de données, et le plan de contrôle [5]. Chacun

de ces éléments joue un rôle essentiel dans le fonctionnement du réseau, permettant ainsi d'assurer sa flexibilité et son efficacité. De plus, ces composants communiquent entre eux via des interfaces bien définies, facilitant la gestion et l'orchestration des différentes fonctions du réseau.

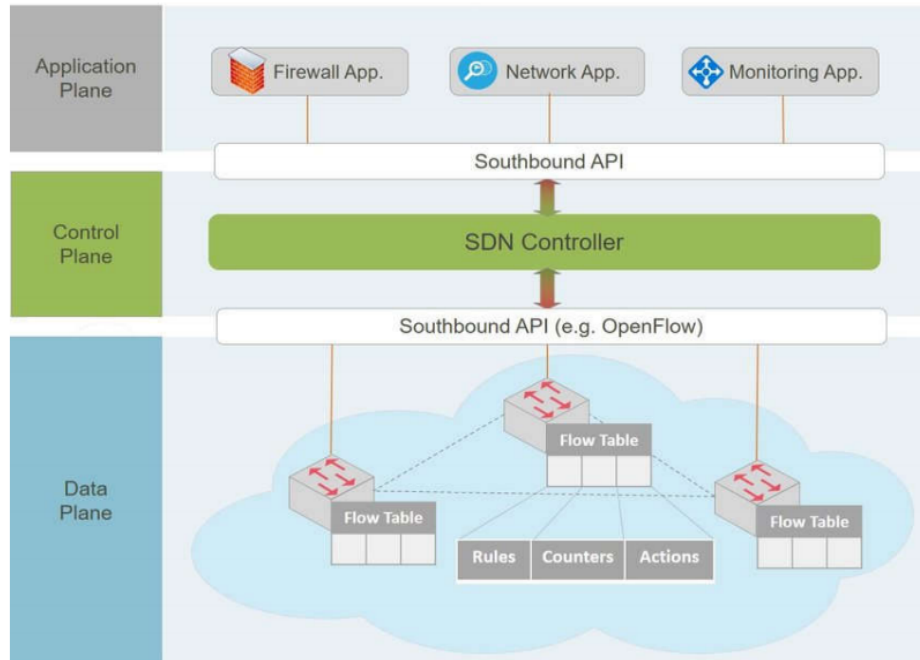


FIGURE II.1

Architecture réseau défini par logiciel [6].

Plan de données (transfert) : Le plan de données, dans un réseau défini par logiciel (SDN), est responsable du transfert réel des données à travers le réseau. Il comprend des équipements tels que des commutateurs et des routeurs qui dirigent les paquets de données d'un point à un autre, en suivant des règles de traitement définies par le plan de contrôle [6]. Ce plan de contrôle est géré par un contrôleur SDN, qui fournit les instructions nécessaires à la gestion du trafic.

Plan de contrôle : est chargé de la gestion et de la prise de décision concernant l'acheminement du trafic réseau. Contrairement au plan de données, qui s'occupe de l'acheminement effectif des paquets, le plan de contrôle détermine la manière dont les données doivent être traitées et dirigées à travers le réseau [7]. Il est généralement centralisé et contrôlé par un contrôleur SDN, qui configure les équipements du plan de données en fonction des règles et des politiques réseau. Le contrôleur communique avec les commutateurs et routeurs pour définir les chemins de données, et s'assure que les décisions relatives à la gestion du trafic sont appliquées de manière cohérente à travers l'infrastructure du réseau.

Plan d'applications : Le plan d'application dans un réseau défini par logiciel (SDN) représente la couche la plus élevée de l'architecture SDN [8] . Il englobe les applications qui interagissent avec le contrôleur pour définir les politiques de gestion du réseau. Ces applications sont responsables de l'optimisation des ressources réseau et de la fourniture de services adaptés aux besoins des utilisateurs finaux, comme la gestion du trafic, la sécurité, ou la qualité de service. Elles utilisent des interfaces "northbound" (NBI) pour communiquer avec le contrôleur et pour lui soumettre des exigences en matière de configuration ou de gestion du réseau. Le plan d'application peut inclure des outils comme les gestionnaires de réseaux, les systèmes de surveillance, ou les outils d'orchestration, qui permettent de configurer et d'optimiser en temps réel les ressources du réseau.

II.2.3 Interfaces de communications

Le contrôleur interagit avec les autres couches à travers les interfaces Sud et Nord et avec les autres contrôleurs à travers une interface Est-Ouest.

Interfaces Sud : les interfaces Sud sont des moyens de communication qui permettent au contrôleur SDN de gérer et de configurer les équipements réseau, tels que les commutateurs et les routeurs, dans le plan de données [9] . Le protocole OpenFlow est le plus utilisé pour ces interfaces, facilitant la transmission de règles de flux entre le contrôleur et les équipements. Lorsqu'un paquet arrive dans un commutateur, ce dernier vérifie son entête et compare les informations avec les règles présentes dans sa table de flux. En cas de non-correspondance, le commutateur envoie une demande au contrôleur pour déterminer l'action appropriée, et le contrôleur renvoie une règle pour traiter ce flux à l'avenir [10] . En plus d'OpenFlow, d'autres protocoles, comme ForCES et SoftRouter, sont également utilisés pour assurer la communication entre le plan de contrôle et le plan de données, offrant ainsi des solutions alternatives pour la gestion du trafic réseau dans les architectures SDN.

Les interfaces Nord : les interfaces Nord sont des mécanismes permettant d'interagir avec le réseau en exploitant l'abstraction du réseau fournie par le plan de contrôle [11] . Elles fonctionnent essentiellement comme des API (Interfaces de Programmation d'Application) plutôt que comme des protocoles traditionnels de gestion de réseau. À l'heure actuelle, aucun standard n'est encore défini entre la couche de contrôle et la couche applicative, que ce soit par l'ONF ou d'autres organisations. L'ONF précise que plusieurs niveaux d'abstraction peuvent être créés, offrant ainsi des interfaces Nord adaptées à différents cas d'utilisation. Parmi les propositions industrielles, une des solutions les plus

courantes est l'API REST (REpresentational State Transfer), qui permet une interface programmable et est largement utilisée dans les applications commerciales, offrant ainsi une manière flexible et évolutive de gérer les réseaux à travers des commandes simples et accessibles.

Est/Ouest : Les interfaces Est/Ouest sont des mécanismes de communication qui facilitent généralement les échanges entre les contrôleurs dans une architecture multi-contrôleurs, afin de synchroniser les états du réseau [12]. Ces architectures étant relativement récentes, il n'existe actuellement aucun standard de communication inter-contrôleur défini.

II.2.4 Protocole OpenFlow

L'origine d'OpenFlow remonte à l'Université de Stanford, où un groupe de chercheurs du Clean Slate Program a entrepris de réfléchir à la conception d'Internet si l'on devait repartir de zéro avec 20 à 30 ans de recul. En 2008, l'Open Networking Foundation (ONF) a publié OpenFlow, un protocole qui permet la mise en œuvre des architectures de réseaux définis par logiciel (SDN) [13]. Ce protocole permet un accès direct au plan de données des équipements réseau tels que les routeurs et les commutateurs, offrant ainsi une méthode dynamique, centralisée et programmable pour interagir avec l'infrastructure réseau [14]. OpenFlow prend en compte les fonctions communes supportées par les tables de flux des commutateurs Ethernet et des routeurs des différents fabricants, ce qui facilite l'utilisation de ces tables de flux dans l'ensemble des dispositifs réseau [15]. Dans les architectures SDN, OpenFlow est principalement utilisé comme interface sud.

Architecture du protocole Openflow : Le protocole OpenFlow établit la communication entre un switch OpenFlow et un contrôleur. Il repose sur un ensemble de messages qui circulent dans les deux sens entre le contrôleur et le switch [16]. Ces messages sont essentiels pour permettre au contrôleur de maintenir un contrôle sur les switches ainsi que sur le trafic des utilisateurs. En ce qui concerne l'architecture du switch OpenFlow, comme évoqué dans la section précédente, celle-ci est régie par les principes des tables de flux.

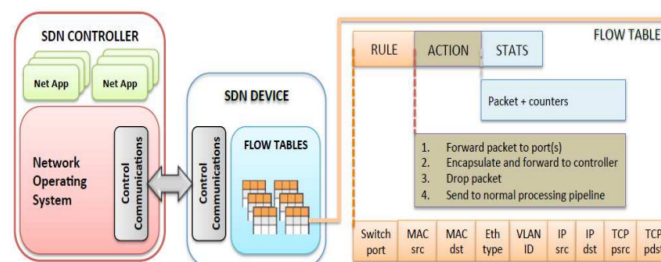


FIGURE II.2
Architecture Openflow [16].

Les messages Openflow

Les messages OpenFlow sont définis par une entête qui contient la longueur, l'identificateur de transaction et le type de message. En connaissant l'adresse IP du contrôleur, le commutateur initie une connexion sécurisée via TLS sur TCP. Ces messages sont classés en trois catégories : symétriques, asymétriques, et Contrôleur-Commutateur [17] .

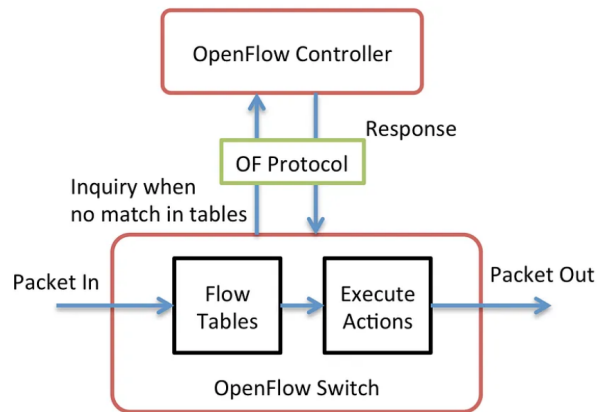


FIGURE II.3
openflow protocole [16]

1. Messages Contrôleur-Commutateur

Ces messages, initiés par le contrôleur, servent à vérifier l'état du commutateur, à gérer sa configuration et à collecter des informations [18] . Ils incluent :

- Features : Permettent au contrôleur de demander l'identité et les capacités du commutateur via une requête.
- Modify-State : Utilisés pour gérer l'état du commutateur. Ils permettent d'ajouter, modifier ou supprimer des entrées dans les tables OpenFlow.
- Configuration : Permettent au contrôleur de définir les paramètres de configuration du commutateur. Ce dernier répond aux requêtes correspondantes.
- Read-State : Utilisés pour collecter des informations sur le commutateur, comme les statistiques, les capacités et la configuration actuelle.
- Packet-Out : Employés pour transférer des paquets reçus via des messages
- Packet-In : Ces messages contiennent soit un paquet entier, soit un ID de buffer se référant à un paquet stocké dans le commutateur. Si aucune action n'est spécifiée, le paquet est détruit.
- Barrier : Permettent au contrôleur de recevoir une notification une fois qu'une opération est terminée

2. Messages asynchrones

Les messages asynchrones, envoyés par le commutateur au contrôleur, signalent des changements d'état ou l'arrivée de nouveaux paquets [18]. Ils comprennent :

- Packet-in : Permet au commutateur de transférer le contrôle d'un paquet au contrôleur.
- Flow-removed : Notifie le contrôleur de la suppression d'une entrée dans la table des flux.
- Port-status : Informe le contrôleur d'un changement d'état sur un port.
- Role-status : Indique qu'un nouveau contrôleur a pris le contrôle, en envoyant un message Role-status au contrôleur concerné.

3. Messages Symétriques

Les messages symétriques sont transmis spontanément, sans sollicitation préalable du commutateur ou du contrôleur [18]. Ils comprennent :

- - Hello : Sert à vérifier la connectivité entre le contrôleur et le commutateur.
- -Echo : Échangés après l'établissement de la connexion, ces messages incluent une requête (ECHO-REQUEST) qui doit être suivie d'une réponse (ECHO-REPLY).
- - Error : Permet au contrôleur ou au commutateur de signaler un problème de connexion.

II.3 Avantages et caractéristiques des réseaux définis par logiciel (SDN)

Le réseau défini par logiciel (SDN) offre une architecture centralisée et programmable, capable de provisionner dynamiquement les ressources réseau pour répondre aux besoins changeants des entreprises [19]. Cette technologie présente plusieurs avantages techniques et commerciaux, notamment

Programmabilité directe : Grâce à la séparation des fonctions de contrôle et de transfert, le SDN permet une configuration réseau automatisée à l'aide d'outils propriétaires ou open source tels qu'OpenStack, Puppet, Salt et Ansible.

Gestion centralisée : L'intelligence du réseau est centralisée dans un contrôleur logiciel qui fournit une vue globale du réseau. Aux applications et moteurs de stratégie, le réseau apparaît comme un commutateur logique unique.

Réduction des coûts : Le SDN réduit le besoin d'investissements dans des équipements réseau spécifiques basés sur ASIC et soutient des modèles de paiement évolutifs. De nombreux commutateurs du marché prennent en charge des solutions comme OpenFlow, adaptées aussi bien aux centres de données qu'à d'autres environnements réseau.

Agilité et flexibilité : Le SDN permet de déployer rapidement de nouvelles applications,

services ou infrastructures pour s'adapter aux besoins changeants des organisations. Une simple mise à jour suffit pour étendre une nouveauté à l'ensemble du réseau.

II.4 Cas d'utilisations du SDN

Les réseaux définis par logiciel (SDN) offrent une gamme étendue d'applications dans les environnements réseau, permettant un contrôle personnalisé, la réduction des équipements intermédiaires et simplifiant le développement et le déploiement de nouveaux services et protocoles. Selon une étude de The Insight Partners [20], le marché des SDN, incluant les équipements d'infrastructure, les logiciels de contrôle et de virtualisation, ainsi que les applications de réseau et de sécurité, devrait atteindre 23,45 milliards de dollars en 2023 et 93,10 milliards de dollars d'ici 2031, avec un taux de croissance annuel composé (TCAC) de 18,8 % entre 2023 et 2031. Cette section explore quelques exemples concrets d'applications des SDN.

Data center et Cloud computing

Les dernières années ont vu l'émergence de la technologie du cloud computing, qui consiste à centraliser les ressources (stockage, calcul, etc.) dans des centres de données et à les offrir en tant que service à l'utilisateur [21]. Cette évolution a entraîné une augmentation massive de la quantité de données traitées et, par conséquent, une explosion du nombre de centres de données. Cela représente un défi pour les réseaux qui gèrent le trafic, notamment pour les grands opérateurs possédant plusieurs centres de données interconnectés. Google, un leader dans ce domaine, a expérimenté l'utilisation du SDN pendant trois ans pour la gestion et le contrôle de ses centres de données répartis dans le monde entier [22]. Le projet a consisté à construire un réseau cœur expérimental, B4, parallèlement à son réseau cœur opérationnel, portant le trafic Internet. B4 relie les centres de données de Google tout en séparant les plans de contrôle et de données. Cela permet une mise en œuvre rapide de nouveaux services et centralise la gestion du trafic via un contrôleur SDN. En conséquence, le débit du réseau a augmenté de 30 % et l'utilisation des ressources réseau a atteint 95 %.

La gestion des ressources informatiques reste l'un des défis majeurs des réseaux Cloud. Le SDN est perçu comme une solution innovante pour faciliter la configuration et la gestion des environnements Cloud et des centres de données. Par exemple, Oracle SDN propose un réseau virtualisé pour les centres de données, connectant dynamiquement des machines virtuelles avec les serveurs réseau. Grâce à l'interface de gestion d'Oracle Fabric, la configuration et la surveillance du réseau virtuel sont possibles à distance, permettant également le déploiement à la demande de services comme les pare-feu, l'équilibrage de

charge et le routage. Selon Oracle, cette approche permet d'augmenter les performances des applications jusqu'à 30 fois.

Multimédia et QOS

L'architecture Internet actuelle repose sur l'envoi des paquets sans prendre en considération le type des paquets ni la qualité de la transmission. Les applications multimédias telles que le streaming vidéo, la vidéo à la demande, la vidéoconférence, la WebTV, etc., nécessitent des ressources réseau stables et tolèrent les erreurs ainsi que les retards de transmission [23]. En s'appuyant sur la vue centralisée du réseau offerte par le SDN, il est possible de sélectionner différents chemins pour les divers flux de trafic en fonction du débit. Dans , les auteurs présentent le Video over Software-Defined Networking (VSDN), une architecture réseau qui détermine la trajectoire optimale en utilisant une vue d'ensemble du réseau. Le VSDN a été implémenté dans NS3 et son comportement a été analysé en prenant en compte la complexité des messages entre le VSDN et les hôtes/switchs (ajout d'une nouvelle table de flux, demande d'un nouveau service, suppression du service), ainsi que la bande passante et les délais de transmission sur le réseau. Le VSDN est un projet de recherche en cours avec d'autres améliorations à apporter.

Réseaux sans fil

Le SDN a également été appliqué dans le domaine des réseaux sans fil, notamment pour la 5G. Selon le rapport d'Ericsson, le trafic mobile a augmenté de 60 % entre 2015 et le premier trimestre 2016. La gestion de cette masse de données constitue un défi majeur pour les opérateurs, car une gestion inadéquate peut entraîner des interférences entre les signaux sans fil et, par conséquent, dégrader la qualité d'expérience (QoE) pour les utilisateurs. Le SDWN (Software Defined Wireless Network) a été proposé comme une solution intégrant le SDN dans les réseaux sans fil pour faciliter la gestion et le déploiement de nouvelles applications dans l'infrastructure réseau. De nouvelles architectures, telles que le CSDN (CellularSDN) [3] et le SoftAir, ont été proposées. Ces architectures collectent des données sur les utilisateurs et les conditions du réseau, qu'elles envoient au contrôleur afin d'optimiser la consommation d'énergie, l'utilisation des ressources et la personnalisation des services pour les utilisateurs.

Le concept de SDN a aussi inspiré des améliorations dans les réseaux domestiques (HAN) et le Big Data . Ce dernier, étant très consommateur de ressources en raison du volume de données traitées, nécessite une infrastructure flexible et bien dimensionnée. Le SDN permet de répondre à ces besoins en ajustant le réseau en temps réel pour optimiser la centralisation des données, accélérer leur traitement et simplifier la gestion des commutateurs.

Sécurité

L'architecture centralisée du SDN permet au contrôleur de détecter rapidement les attaques et de limiter leurs effets. Plusieurs mécanismes ont été proposés pour détecter les attaques par déni de service (DoS et DDoS). Par exemple, dans [24], les auteurs suggèrent d'isoler les périphériques réseau des attaques intra-LAN en utilisant OpenFlow. Dans cette architecture, un composant IDS est chargé de détecter les attaques en identifiant les équipements non sécurisés et d'informer le contrôleur. Les résultats ont montré que cette solution est efficace sur un réseau LAN réel.

Un autre mécanisme de sécurité a été proposé dans [24], visant à inspecter tous les échanges de flux entre les commutateurs et le contrôleur pour prévenir les attaques de type « IP spoofing ». Les résultats de simulation ont montré que ce plan renforce la sécurité en temps réel à différents niveaux, avec une configuration minimale et une faible surcharge sur le contrôleur, car tout le traitement est effectué en dehors de celui-ci. Cela permet de bloquer rapidement les attaques au niveau du contrôleur et des commutateurs et de tracer leurs sources.

II.5 Conclusion

Dans ce chapitre, nous avons exploré en profondeur la technologie des réseaux SDN, en analysant non seulement son architecture et ses composants clés, mais aussi le rôle essentiel du protocole OpenFlow. Nous avons mis en lumière les nombreux atouts de cette technologie, notamment sa capacité à centraliser la gestion du réseau, offrant ainsi une flexibilité inédite pour la configuration et la gestion des ressources. Grâce à sa séparation des plans de contrôle et de données, le SDN facilite l'automatisation, l'orchestration et l'adaptation dynamique du réseau en fonction des besoins. En outre, ses applications s'étendent à divers domaines, permettant des améliorations significatives en termes d'agilité, de performance et de sécurité.

Chapitre III

IMPLÉMENTATION DU RÉSEAU SDN

III.1 Introduction

Dans ce chapitre, nous passons à la mise en pratique des réseaux SDN en décrivant les étapes de leur implémentation. Après avoir présenté les bases théoriques dans le chapitre précédent, nous allons ici expliquer le choix de la plateforme de simulation, la configuration du contrôleur SDN, le déploiement des commutateurs, ainsi que l'intégration des applications nécessaires. Nous terminerons par une description du réseau de test qui servira à évaluer les performances de notre implémentation en utilisant certains métriques tels que la latence, le débit et la perte de paquets, qui jouent un rôle essentiel dans l'analyse de l'efficacité du réseau.

III.2 Emulateur et contrôleur SDN

Pour simuler notre réseau SDN, nous avons utilisé Mininet comme émulateur pour simuler et tester des topologies SDN, offrant la possibilité de reproduire un réseau réel avec des commutateurs, des hôtes et des connexions. Pour le plan de contrôle, nous avons choisi ONOS, un contrôleur SDN open source reconnu pour sa scalabilité et ses fonctionnalités avancées, permettant une gestion efficace des réseaux. L'installation d'ONOS a été réalisée via Docker, qui simplifie le déploiement en isolant les environnements et en réduisant les conflits de dépendances. La figure III.1 montre une idée claire sur la relation entre eux. L'ensemble de l'émulation a été mis en œuvre sur une machine virtuelle Ubuntu hébergée dans VirtualBox, offrant un environnement sécurisé, flexible et reproductible pour nos expérimentations.

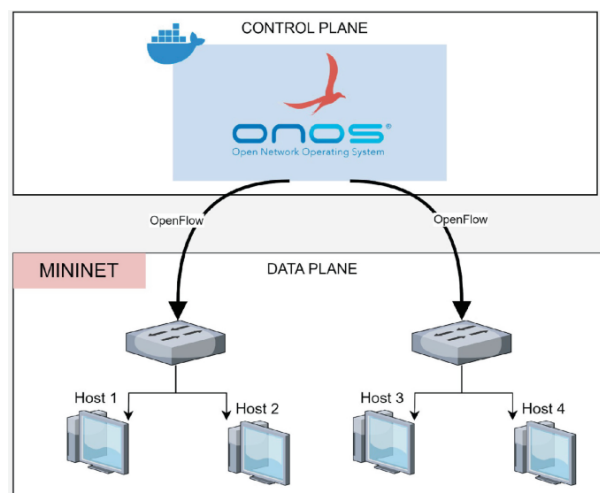


FIGURE III.1
plateforme de simulation [25].

III.2.1 Emulateur Mininet

Mininet est un émulateur de réseau qui permet de créer un réseau virtuel composé d'hôtes, de commutateurs, de contrôleurs et de liens. Les hôtes utilisent le logiciel réseau standard de Linux, tandis que les commutateurs prennent en charge le protocole OpenFlow, offrant ainsi une grande flexibilité pour expérimenter avec le routage et les réseaux définis par logiciel (SDN) [26]. Pour simplifier la création et la personnalisation des topologies, Mininet propose MiniEdit, une interface graphique qui permet de concevoir facilement des réseaux en glissant-déposant des éléments. Mininet est ainsi largement utilisé pour la recherche, le développement, le prototypage, le débogage et l'apprentissage, car il fournit un environnement réaliste pour tester et évaluer des concepts réseaux.

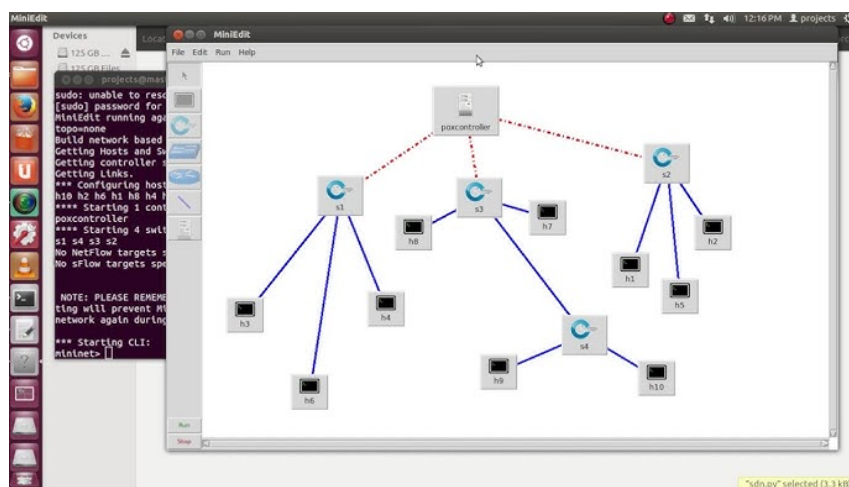


FIGURE III.2
interface graphique MiniEdit [26].

III.2.2 Contrôleur Open Networking Operating System (ONOS)

ONOS (Open Network Operating System) est un système d'exploitation réseau open source conçu pour le contrôle et la gestion des réseaux définis par logiciel (SDN) [27]. Ce contrôleur SDN facilite la création d'applications réseau de nouvelle génération en permettant aux utilisateurs de contrôler le réseau en temps réel sans la nécessité de commutateurs physiques, ce qui simplifie et réduit les coûts des tests de protocoles. ONOS permet de gérer une variété de périphériques réseau, offrant ainsi une flexibilité et une scalabilité pour les infrastructures réseaux.

Son architecture ouverte et sa capacité à fonctionner dans le cloud ouvrent la voie à des innovations importantes dans la gestion des réseaux [28]. En tant que logiciel open source, ONOS permet aux utilisateurs et aux entreprises de développer des solutions personnalisées pour répondre aux besoins spécifiques des réseaux définis par logiciel.

Le système est soutenu par la Fondation ONOS et est largement utilisé par des acteurs

majeurs du secteur des télécommunications et des réseaux.

Caractéristiques d'ONOS : ONOS est conçu pour ces caractéristiques :

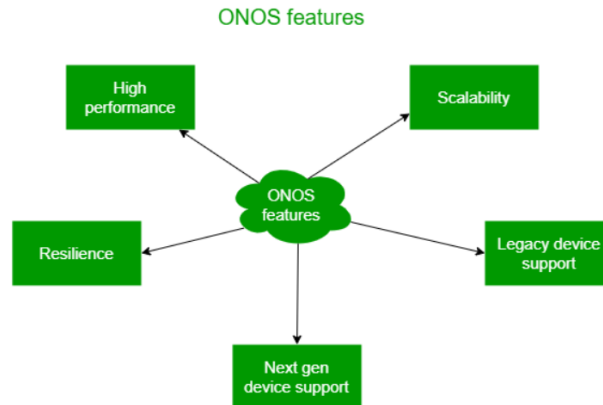


FIGURE III.3
Caractéristiques d'ONOS [27] .

- **Haute performance** : Il exécute et répond aux spécifications exactes des opérateurs de réseau. Toutes les versions d'ONOS visent principalement la haute performance. Il prend en charge des millions d'intentions avec un temps de réponse étonnant de seulement 50 ms.
- **Évolutivité** : Il peut évoluer aussi haut que nécessaire en ajoutant de nouvelles instances. Nous pouvons également bénéficier d'une prise en charge de passerelle évolutive dans ONOS, ouvrant ainsi la voie à l'équilibrage de charge et à la haute disponibilité. Une passerelle évolutive garantit une haute disponibilité en autorisant plusieurs passerelles redondantes vers le système. Elle permet également l'ajout ou la suppression dynamique de nœuds de passerelle et la prise en charge d'une défaillance de passerelle.
- **Résilience** : ONOS fournit à l'utilisateur des réseaux d'opérateurs critiques pour sa mission. Grâce à ONOS, nous pouvons mettre en œuvre des réseaux résilients contrôlés par logiciel (ReCSDN). Il peut être déployé au-dessus d'ONOS pour confirmer la viabilité de l'approche.
- **Prise en charge des appareils existants** : Avec ONOS, nous pouvons facilement ajouter ou configurer les appareils traditionnels. Nous pouvons ajouter des services avec une configuration dynamique basée sur un modèle. Grâce à ONOS, nous pouvons facilement connecter des appareils SDN et existants. Il peut s'adapter à n'importe quel appareil existant.
- **Prise en charge des périphériques de nouvelle génération** : ONOS permet un contrôle en temps réel des périphériques de plan de données SDN natifs et peut

désormais être utilisé avec la prise en charge p4. ONOS développe également une pile open source pour fournir aux utilisateurs la configuration et le contrôle du réseau.

Gestion et Visualisation du Réseau via l'Interface GUI d'ONOS : Le contrôleur ONOS possède une interface graphique utilisateur (GUI) complète et intuitive, conçue pour simplifier la gestion des réseaux définis par logiciel (SDN) [27]. Cette interface permet aux administrateurs de visualiser en temps réel l'état du réseau, y compris la topologie des périphériques, les flux de données et les intentions réseau. Grâce à cette interface, il est possible de gérer facilement les périphériques, d'ajouter ou de supprimer des flux, de superviser les performances du réseau, et de configurer des intentions de réseau de manière centralisée. ONOS permet également de surveiller l'état du cluster et d'effectuer des actions de gestion sur les nœuds du système. En offrant une vue d'ensemble du réseau et des outils de gestion détaillés, l'interface GUI d'ONOS facilite l'administration des réseaux SDN, en améliorant l'efficacité et la réactivité des opérateurs.

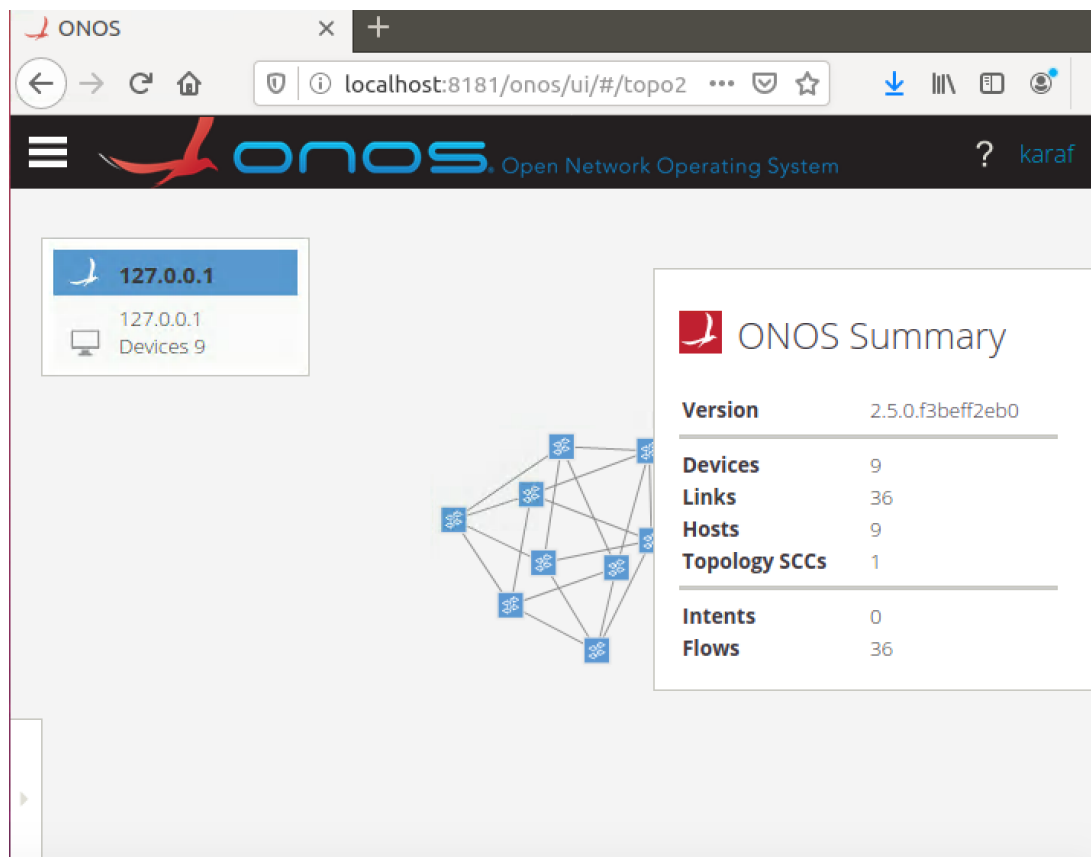


FIGURE III.4
Interface Graphique Utilisateur (GUI) d'ONOS [27] .

III.2.3 Docker

Docker est un outil de conteneurisation qui permet d’empaqueter une application et ses dépendances dans un conteneur isolé, capable de s’exécuter de manière fiable sur n’importe quel environnement. Contrairement à la virtualisation, la conteneurisation s’appuie sur les ressources de la machine hôte, ce qui la rend plus légère et efficace.

Dans le contexte d’ONOS, Docker offre une solution pratique pour déployer et tester rapidement le contrôleur dans différents environnements. Grâce à Docker, il est possible de créer des conteneurs ONOS qui encapsulent toutes les dépendances nécessaires, éliminant ainsi les problèmes liés aux incompatibilités entre systèmes [29]. Cette approche garantit une portabilité accrue, permettant à ONOS de fonctionner de manière fiable, que ce soit sur une machine locale, un cloud privé ou public. De plus, Docker facilite la création de topologies réseau complexes en utilisant plusieurs instances ONOS dans des conteneurs distincts, tout en simplifiant la gestion des ressources et des configurations.

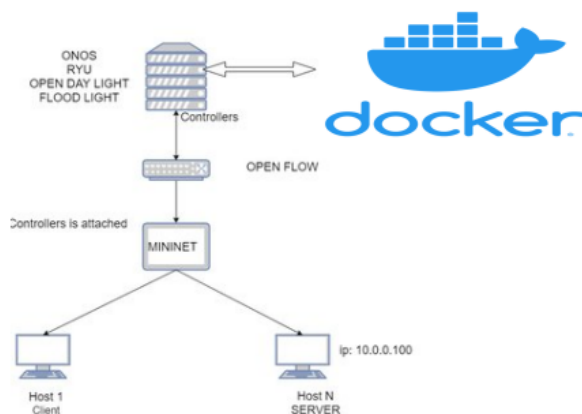


FIGURE III.5

Interopérabilité d’ONOS avec Docker dans un Environnement SDN [30].

III.2.4 Open vSwitch

Open vSwitch est un commutateur virtuel multicouche de qualité de production sous licence open source Apache 2.0 . Il est conçu pour permettre une automatisation massive du réseau via une extension programmatique, tout en prenant en charge les interfaces et protocoles de gestion standard (par exemple NetFlow, sFlow, IPFIX, RSPAN, CLI, LACP, 802.1ag). En outre, il est conçu pour prendre en charge la distribution sur plusieurs serveurs physiques, à l’instar du commutateur virtuel distribué vNetwork de VMware ou du Nexus 1000V de Cisco.

Dans le cadre de notre projet, nous avons utilisé Open vSwitch comme commutateur virtuel pour simuler un réseau SDN [31]. Grâce à sa compatibilité avec le protocole Open-Flow, OVS permet un contrôle centralisé des flux réseau via un contrôleur SDN comme ONOS. Cette compatibilité facilite la gestion dynamique des routes et la mise en œuvre de politiques réseau complexes, essentielles pour les environnements SDN.

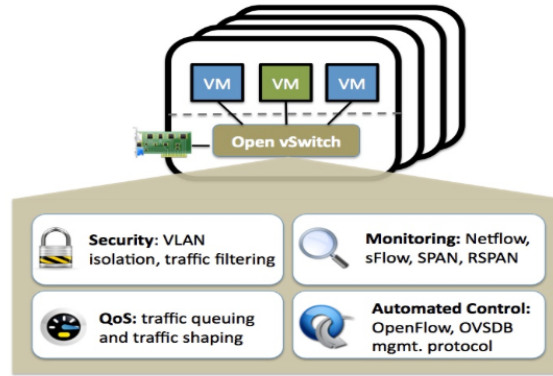


FIGURE III.6

Commutateur virtuel Open vSwitch utilisé dans la simulation SDN [31].

III.3 Outils d'évaluation

Les outils d'évaluation de réseau sont essentiels pour analyser, surveiller et optimiser les performances des infrastructures réseau. Ils permettent de mesurer des paramètres tels que la bande passante, la latence, la perte de paquets, et la qualité de service, tout en offrant une vue détaillée du comportement des données à travers le réseau et parmi les outils on a iperf .

iPerf est un outil de mesure de performance réseau utilisé pour tester la bande passante et évaluer la qualité d'une connexion réseau entre deux hôtes. Il permet de générer des flux de données et d'analyser divers paramètres comme la bande passante, la latence, la perte de paquets [32]. iPerf peut être utilisé pour tester des connexions TCP, UDP, et SCTP sur des réseaux IPv4 et IPv6.

iPerf features :

iPerf permet d'extraire une large gamme de métriques pour évaluer en profondeur les performances d'un réseau. Ces métriques sont cruciales pour analyser le comportement du réseau sous différents protocoles (TCP, UDP) et dans divers scénarios de test.

- **Bande passante** : Mesure de la quantité de données qui peuvent être transmises sur le réseau pendant une période donnée.

- **Latence (RTT)** : Temps nécessaire pour qu'un paquet de données fasse un aller-retour entre deux points du réseau (Round Trip Time).
- **Perte de paquets** : Pourcentage de paquets envoyés qui n'arrivent pas à leur destination.
- **Taille de la fenêtre TCP** : Quantité de données qui peuvent être envoyées avant d'attendre un accusé de réception, influençant la performance TCP.
- **Débit global (TCP/UDP)** : Mesure du débit de données total envoyé et reçu par le réseau, mesuré pour les protocoles TCP et UDP.
- **Total d'octets envoyés et reçus** : Quantité totale de données envoyées et reçues durant le test.
- **Durée du test** : La durée du test fait référence à l'intervalle de temps durant lequel les tests de performance d'un réseau sont effectués. Elle permet d'évaluer de manière précise le comportement d'un réseau sur une période donnée, afin d'observer les tendances et de mesurer des métriques de performance, telles que la latence, le débit ou la fiabilité.
- **Taux d'erreurs** : Fréquence des erreurs dans la transmission des données, comme les erreurs de bits ou de paquets.

III.4 Outils de Connexion et de Gestion à Distance

Dans le cadre de ce projet, PuTTY et Xming Server ont été utilisés pour établir une connexion entre la machine virtuelle Ubuntu et la machine hôte Windows, tout en permettant l'exécution d'applications graphiques de la machine virtuelle sur le système hôte.

III.4.1 PuTTY

PuTTY est un client SSH (Secure Shell) open source qui permet une connexion sécurisée à distance vers des serveurs ou des machines virtuelles. Il a été utilisé pour accéder à la machine virtuelle Ubuntu via le protocole SSH, offrant une interface terminal simple et efficace. L'utilisation de PuTTY permet la gestion des services et le contrôle des processus sur le système distant de manière sécurisée.

III.4.2 Xming Server

Xming Server est un serveur X Window pour Windows, permettant d'afficher des applications graphiques d'une machine distante exécutant Linux ou Unix. Dans ce projet,

il a été configuré pour exécuter des applications graphiques hébergées sur une machine virtuelle Ubuntu. Cela permet à l'utilisateur d'interagir avec des applications Linux depuis un système Windows, facilitant ainsi les tests et la gestion des fonctionnalités liées à l'interface utilisateur graphique. En utilisant Xming, il devient possible d'exécuter des applications graphiques Linux tout en restant dans un environnement Windows, simplifiant ainsi l'interaction avec des outils distants sans nécessiter une configuration complexe.

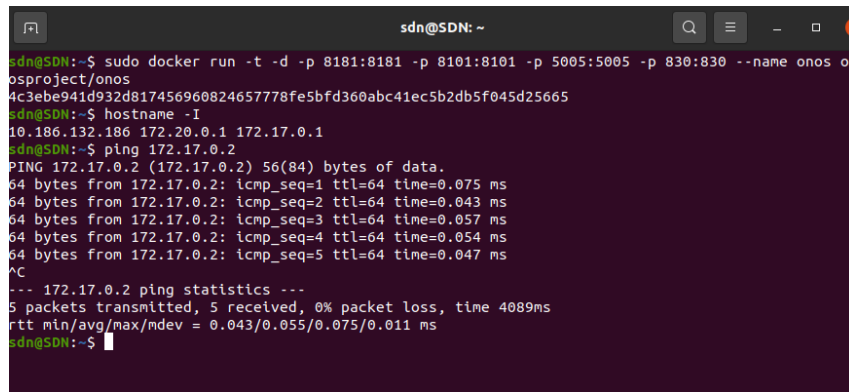
III.5 Configuration de l'environnement

Pour configurer notre environnement SDN, nous avons suivi une série d'étapes essentielles pour garantir une installation fonctionnelle et efficace et parmi les étapes on a :

- **Installation de la machine virtuelle** : Nous devons configurer un environnement Linux où nous exécuterons ONOS. Pour cela, nous utilisons une image Ubuntu 18.04.3 et l'importons dans VirtualBox en suivant quelques étapes de configuration de base.
- **Installation de Docker** : Nous installons Docker pour faciliter l'installation des containers nécessaires à ONOS.
- **Installation d'ONOS avec Docker** : Une fois Docker installé, nous pouvons procéder à l'installation d'ONOS. Après l'installation, certaines applications doivent être activées via l'interface graphique d'ONOS pour configurer l'environnement avant de créer des topologies.
- **Installation de Mininet** : Mininet est nécessaire pour créer des topologies avec ONOS. Nous l'installons en utilisant la commande "apt-get" sur le système Linux, sans avoir besoin de Docker. Une fois installé, nous pouvons créer une topologie simple composée de deux hôtes pour tester les fonctionnalités d'ONOS avec Mininet.

III.5.1 Démarrage d'ONOS :

lancer le conteneur ONOS, obtenir l'adresse de la machine virtuelle, puis tester la connectivité vers le contrôleur ONOS dont l'adresse est 172.17.0.2.



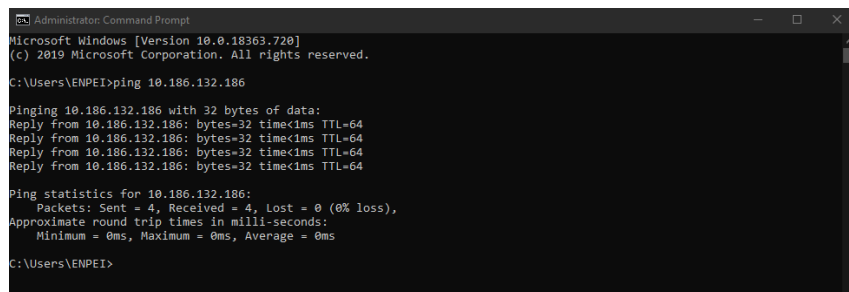
```
sdn@SDN: ~
sdn@SDN:~$ sudo docker run -t -d -p 8181:8181 -p 8101:8101 -p 5005:5005 -p 830:830 --name onos on
osproject/onos
4c3ebe941d932d817456960824657778fe5bfd360abc41ec5b2db5f045d25665
sdn@SDN:~$ hostname -I
10.186.132.186 172.20.0.1 172.17.0.1
sdn@SDN:~$ ping 172.17.0.2
PING 172.17.0.2 (172.17.0.2) 56(84) bytes of data:
64 bytes from 172.17.0.2: icmp_seq=1 ttl=64 time=0.075 ms
64 bytes from 172.17.0.2: icmp_seq=2 ttl=64 time=0.043 ms
64 bytes from 172.17.0.2: icmp_seq=3 ttl=64 time=0.057 ms
64 bytes from 172.17.0.2: icmp_seq=4 ttl=64 time=0.054 ms
64 bytes from 172.17.0.2: icmp_seq=5 ttl=64 time=0.047 ms
^C
--- 172.17.0.2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4089ms
rtt min/avg/max/mdev = 0.043/0.055/0.075/0.011 ms
sdn@SDN:~$
```

FIGURE III.7

Test de connectivité entre la machine virtuel et le conteneur ONOS .

III.5.2 Test de la connectivité vers la vm

Pour contrôler la machine virtuelle ayant l'adresse IP 10.186.132.186 depuis la machine hôte sous Windows, nous avons d'abord testé la connectivité en exécutant un ping vers l'adresse IP de la machine virtuelle. Cette étape est essentielle pour vérifier que la communication entre la machine hôte et la machine virtuelle est correctement établie. Les résultats du test de ping sont illustrés dans la Figure III.8



```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.18363.720]
(c) 2019 Microsoft Corporation. All rights reserved.

C:\Users\ENPEI>ping 10.186.132.186

Pinging 10.186.132.186 with 32 bytes of data:
Reply from 10.186.132.186: bytes=32 time<1ms TTL=64
Reply from 10.186.132.186: bytes=32 time<1ms TTL=64
Reply from 10.186.132.186: bytes=32 time<1ms TTL=64
Reply from 10.186.132.186: bytes=32 time<1ms TTL=64

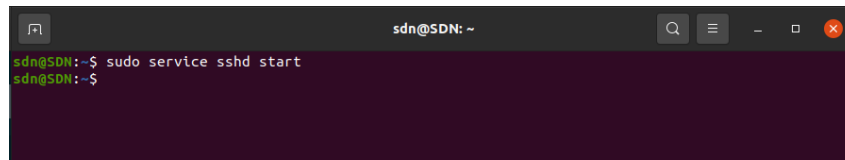
Ping statistics for 10.186.132.186:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\Users\ENPEI>
```

FIGURE III.8

Résultat de la commande ping entre la machine hôte et la machine virtuelle.

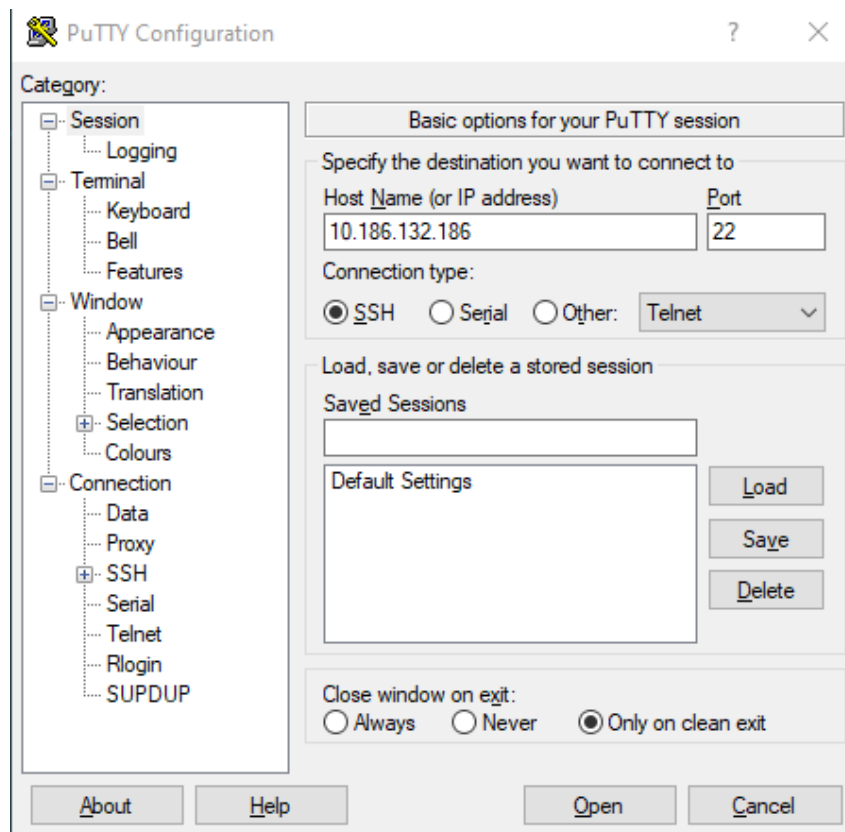
Après l'exécution de la commande présentée dans la, le service SSH (sshd) est lancé sur la machine Ubuntu, ce qui permet à la machine d'accepter des connexions SSH. Cela signifie que vous pouvez désormais vous connecter à distance à cette machine via PuTTY en utilisant son adresse IP et en vous authentifiant avec les identifiants appropriés. En d'autres termes, après avoir démarré le service SSH, vous pouvez établir une connexion sécurisée à votre machine Ubuntu depuis une autre machine, facilitant ainsi la gestion à distance et l'accès à vos ressources sans avoir à être physiquement présent devant la machine.

**FIGURE III.9**

Configuration de la connexion SSH entre la machine hôte et la machine virtuelle.

III.5.3 Etablissement de connexion ssh entre la machine hôte et la machine virtuelle

Pour configurer PuTTY, commencez par ouvrir l'application et entrez l'adresse IP récupérée dans le champ correspondant. Sélectionnez ensuite "SSH" comme type de connexion. Si des options supplémentaires sont nécessaires, telles que l'activation du forwarding X11 pour permettre l'exécution d'applications graphiques via Xming, configurez-les en conséquence. Un exemple de configuration d'une session PuTTY avec ces paramètres est illustré dans la Figure III.10.

**FIGURE III.10**

Configuration de PuTTY avec redirection X11 pour Xming.

Après cela, nous serons capables d'accéder à la ligne de commande de la machine virtuelle (VM) via la connexion SSH, ce qui nous permettra d'exécuter des commandes à

distance et de gérer la VM de manière sécurisée.

```
sdn@SDN: ~
login as: sdn
sdn@10.186.132.186's password:
Welcome to Ubuntu 20.04.3 LTS (GNU/Linux 5.15.0-139-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

130 updates can be applied immediately.
1 of these updates is a standard security update.
To see these additional updates run: apt list --upgradable

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

Your Hardware Enablement Stack (HWE) is supported until April 2025.
Last login: Sat May  2 12:31:11 2026 from 192.168.1.6
sdn@SDN:~$
```

FIGURE III.11

Interface CLI de la machine virtuelle via une connexion SSH sécurisée.

III.5.4 Validation de la configuration de l'environnement SDN via une simulation de topologie

Pour vérifier la bonne configuration de notre environnement SDN, nous avons développé un script Python utilisant Mininet pour simuler une topologie réseau. Cette topologie est constituée de 30 commutateurs de type Open vSwitch (OVS) et de 50 hôtes, connectés de manière linéaire. Les commutateurs sont configurés pour fonctionner avec le protocole OpenFlow, ce qui leur permet de communiquer avec le contrôleur SDN ONOS, déployé à distance à l'adresse IP 172.17.0.2 sur le port 6633. Le contrôleur est ajouté au réseau à l'aide de la classe RemoteController fournie par Mininet.

```
sdn@SDN: ~/Downloads
1 of these updates is a standard security update.
To see these additional updates run: apt list --upgradable

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

Your Hardware Enablement Stack (HWE) is supported until April 2025.
Last login: Sat May  2 12:31:11 2026 from 192.168.1.6
sdn@SDN:~$ cd Downloads/
sdn@SDN:~/Downloads$ sudo python3 fixedSdn_enhanced\1\)'
[sudo] password for sdn:

=====
Batch 1/12 - 5 active hosts
=====
[*] ONOS flows flushed on 0 device(s).
[*] Settling for 5s ...
[*] Warming up flows ...
[+] Same Switch (0 hops) h00 -> h01 avg RTT = 0.050 ms
    iperf (UDP bw=10.50 Mbps loss=0.00%)
[+] One Hop (1 hop) h00 -> h03 avg RTT = 0.861 ms
    iperf (UDP bw=10.50 Mbps loss=0.00%)
[+] Furthest Switch h00 -> h04 avg RTT = 1.442 ms
```

FIGURE III.12

Exécution d'un script Python pour créer une topologie réseau SDN dans Mininet.

Après avoir exécuté le fichier Python contenant le script de simulation, la topologie ré-

seau est automatiquement détectée et affichée dans l'interface graphique (GUI) d'ONOS. Cette interface, accessible via un navigateur web, offre une représentation visuelle intuitive des commutateurs, des hôtes et des liens établis dans le réseau simulé. La figure III.13 montre un aperçu de la topologie affichée dans l'interface graphique d'ONOS après exécution du script Python.

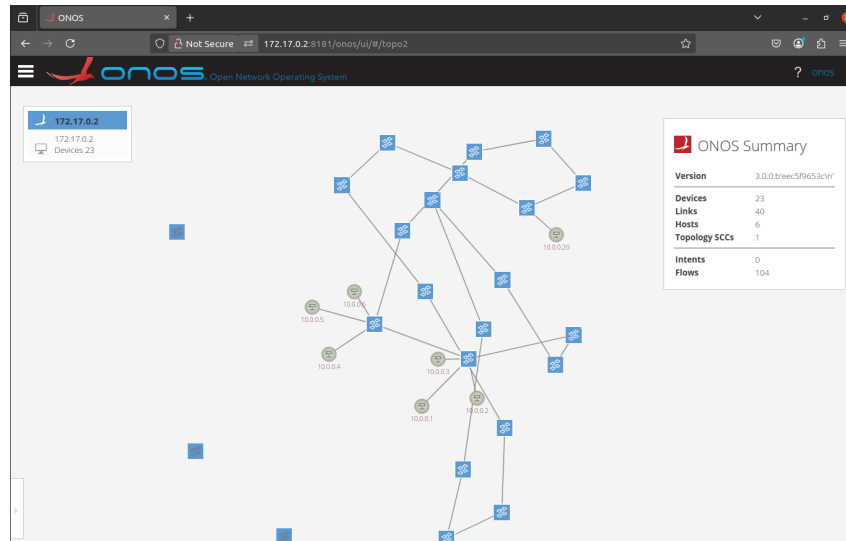


FIGURE III.13

Topologie réseau détectée et affichée dans l'interface graphique d'ONOS.

Pour tester et interagir avec la topologie, le script offre une interface en ligne de commande (CLI - Command Line Interface) permettant d'exécuter diverses commandes de diagnostic, telles que la vérification de la connectivité ou l'exécution de tests de performance. Par exemple, nous avons utilisé la commande `pingall`, qui vérifie la connectivité entre tous les hôtes de la topologie.

III.6 Conclusion

En conclusion, ce chapitre a porté sur la création d'un environnement SDN fonctionnel, intégrant les outils et technologies nécessaires tels que Mininet, ONOS et Docker. Les étapes décrites ont permis de configurer l'infrastructure de base, notamment la mise en place du contrôleur, des commutateurs virtuels et des connexions réseau, tout en assurant une interopérabilité entre les différents composants. Bien que ce travail ait essentiellement consisté à établir cet environnement, il constitue une étape cruciale pour la suite de notre projet. Cet environnement offre désormais un cadre stable et opérationnel qui pourra être utilisé dans les prochains chapitres pour mener des analyses approfondies, effectuer des tests de performance, et explorer les potentialités des réseaux définis par logiciel.

Chapitre IV

ÉVALUATION DES PERFORMANCES DU RÉSEAU SDN

IV.1 Introduction

Après avoir exploré en détail les principes fondamentaux des réseaux définis par logiciel (SDN) et analysé les différentes méthodes de gestion et de contrôle du trafic, nous passons à l'évaluation des performances du réseau SDN. Cette section se concentre sur l'analyse de différents scénarios de test, notamment avec un plan de contrôle centralisé, afin d'évaluer l'impact de ce architecture sur les performances du réseau. Nous comparons également ce configuration avec un réseau traditionnel, où la gestion du trafic est décentralisée, sans l'intervention d'un contrôleur central. Les métriques clés pour cette évaluation incluent la latence, le débit et la bande passante, permettant d'apprécier l'efficacité et la réactivité du réseau dans divers contextes d'utilisation. Les résultats obtenus fourniront une compréhension approfondie de l'impact des différentes architectures de contrôle sur la performance du réseau SDN par rapport à un réseau classique.

IV.2 Méthodologie

Cette partie décrit la méthodologie employée pour évaluer et comparer les performances de différents types de réseaux : réseaux traditionnel et réseaux SDN centralisé . L'objectif principal est d'analyser l'impact de l'architecture SDN sur les performances du réseau en utilisant des métriques clés telles que la bande passante, la latence et la perte de paquets.

IV.2.1 Topologies des réseaux étudiés

Pour atteindre cet objectif, une approche basée sur la simulation a été adoptée. Plusieurs scénarios de réseau ont été définis, chacun correspondant à une architecture spécifique et adaptée au type de réseau étudié. Ces scénarios permettent de comparer les performances des réseaux non-SDN et SDN sous différentes configurations.

- Mode Legacy (Réseaux traditionnel) : Ce scénario représente une architecture réseau classique dans laquelle chaque équipement réseau combine à la fois les fonctions de contrôle et de traitement des données. Il fonctionne sans l'intervention d'un contrôleur SDN centralisé.
- Mode Centralisé (un seul contrôleur SDN) : Ce scénario met en œuvre une architecture SDN basée sur un contrôleur centralisé. Le contrôleur gère l'ensemble des flux de données et la configuration des équipements réseau.

IV.2.2 Métriques de comparaison

De nombreuses métriques peuvent être utilisées pour évaluer les performances des réseaux, notamment la bande passante, le jitter, la latence (RTT), la perte de paquets, ainsi que le total d'octets envoyés et reçus.

Dans cette étude, trois métriques principales ont été retenues, car elles jouent un rôle déterminant dans l'évaluation globale des réseaux :

- La bande passante : Elle mesure la capacité maximale de transmission des données dans le réseau. Cette métrique est essentielle pour évaluer l'efficacité du réseau à gérer un volume important de trafic, en particulier dans des environnements à haute charge tels que les applications IoT.
- La latence (RTT) : La latence correspond au temps nécessaire pour qu'un paquet effectue un aller-retour entre la source et la destination. Cette métrique est cruciale pour les applications sensibles au temps, comme le streaming en temps réel ou les communications critiques.

Le choix de ces métriques met en lumière les aspects les plus critiques des performances réseau et permet de comparer efficacement les différentes architectures étudiées (non-SDN, SDN centralisé).

IV.2.3 Scénarios de simulation et collecte des données

Dans cette section, nous décrivons le processus d'extraction des données réseau et la méthodologie utilisée pour simuler différents scénarios. L'objectif est d'étudier l'impact des variations de topologie et des charges réseau sur les métriques sélectionnées (bande passante, latence et perte de paquets).

- **Variation de la topologie réseau** : Pour évaluer la scalabilité des réseaux étudiés (traditionnel, centralisé), la même topologie a été utilisée dans les trois scénarios afin d'assurer une comparaison équitable. Les conditions réseau, telles que la charge réseau et les paramètres de scalabilité, ont été maintenues identiques. Des topologies réseau de tailles croissantes ont été générées en augmentant progressivement le nombre de nœuds (hôtes) connectés au réseau et en ajoutant des switches intermédiaires. Ces ajustements permettent d'analyser comment les performances évoluent dans des environnements similaires à mesure que le réseau devient plus complexe, tout en mettant en évidence les différences entre les trois types d'architectures.
- **Variation de la charge réseau** : Afin de simuler des environnements proches de la réalité, différents volumes de données ont été générés et envoyés simultanément par plusieurs hôtes. Cette approche repose sur l'utilisation d'**iPerf** pour mesurer

la bande passante et générer du trafic réseau, l'envoi simultané de données par plusieurs nœuds en utilisant des **threads** pour recréer une surcharge réseau réaliste, et l'analyse des variations des métriques en fonction de la quantité de données échangées et du nombre de connexions simultanées.

- **Extraction des données réseau** : Pour extraire les données réseau, plusieurs outils ont été utilisés pour capturer les métriques essentielles comme la bande passante et la latence. Ces outils ont été automatisés dans des scripts pour collecter les résultats à chaque itération, en augmentant le nombre de nœuds et en simulant différentes charges réseau. Voici les commandes utilisées pour chaque métrique :

- **Capture de la bande passante (TCP)** : Pour mesurer la bande passante en utilisant le protocole TCP, j'ai exécuté **iPerf** avec les commandes suivantes :
Côté serveur (TCP) :

```
iperf -s
```

-s : Démarre le serveur **iPerf** qui attend les connexions du client.

Côté client (TCP) :

```
iperf -c <adresse_ip_du_serveur> -t 10
```

-c <adresse_ip_du_serveur> : Définit l'adresse IP du serveur.

-t 10 : Durée du test en secondes (ici 10 secondes).

- **Capture de la latence (ICMP)** : Pour mesurer la latence des paquets ICMP entre le client et le serveur, j'ai utilisé **ping** avec la commande suivante :

```
ping -c <num_requests> -i 0.1 <adresse_IP_dst>
```

-c <num_requests> : Définit le nombre de requêtes **ping** à envoyer.

-i 0.1 : Intervalle de 0.1 seconde entre chaque requête **ping**.

<adresse_IP_dst> : L'adresse IP de la destination pour mesurer la latence.

- **Automatisation des tests** : L'exécution des commandes pour chaque métrique a été automatisée à l'aide de scripts, permettant ainsi de tester les trois types de réseaux étudiés (réseaux traditionnel et réseaux SDN centralisé). Ces scripts ont été conçus pour augmenter progressivement le nombre de nœuds (hôtes et switches) à chaque itération, afin de tester la scalabilité des réseaux dans différents scénarios. Ils varient également la taille des paquets et la charge réseau, simulant ainsi des scénarios réalistes tels que ceux observés dans les réseaux IoT, où la demande de bande passante peut augmenter rapidement. Pour simuler des conditions réalistes de surcharge réseau, des threads ont été utilisés pour envoyer simultanément des données entre chaque paire client-serveur **iPerf**, notamment pour mesurer la bande

passante avec des capteurs. Cela permet de mieux simuler le comportement des réseaux sous charges élevées. Les résultats des tests sont sauvegardés dans des fichiers de csv, facilitant ainsi l'analyse ultérieure des performances des réseaux dans les différents scénarios. L'implémentation exacte de ces scripts est détaillée dans l'annexe du rapport.

Ces scripts ont permis de mesurer et de sauvegarder les données relatives à la bande passante, la perte de paquets et la latence, tout en simulant une charge réseau croissante et en observant les effets sur les trois types de réseaux (non-SDN, SDN centralisé).

- **Processus expérimental** : Le processus global se déroule comme suit :
 - Création d'une topologie réseau dans **Mininet**, adaptée au scénario à tester.
 - Génération du trafic réseau avec **iPerf**, tout en augmentant progressivement la complexité (topologie et charge).
 - Enregistrement des métriques mesurées dans des fichiers pour chaque itération de simulation.

Cette méthodologie garantit une analyse exhaustive des performances réseau en fonction de la taille et de la charge du réseau, tout en fournissant des données fiables pour la comparaison des trois types d'architectures étudiées.

IV.2.4 Résultats et Analyse

Cette partie du rapport présente et analyse les résultats obtenus lors des simulations visant à évaluer les performances réseau en fonction de différentes architectures.

- **Bande passante** : Dans cette section, nous analysons l'impact du nombre d'hôtes sur la bande passante moyenne dans différents scénarios : avec un seul contrôleur SDN(mode centralisé) et dans un réseau traditionnel sans contrôleur(mode legacy). Cette comparaison permet de mettre en évidence les forces et les limitations de chaque architecture en fonction de la charge réseau.

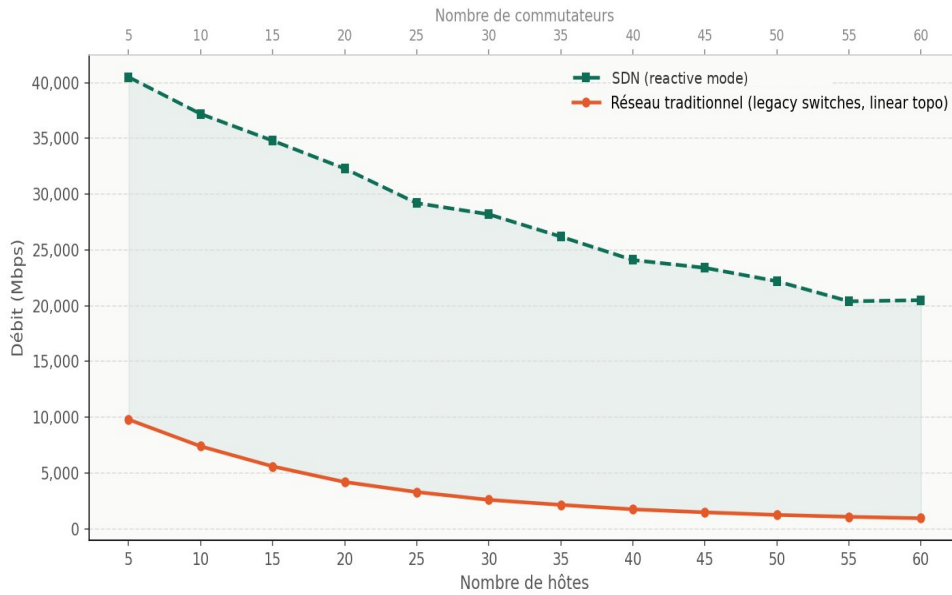


FIGURE IV.1

Variation du débit en fonction du nombre de hôtes

Les résultats présentés dans la figure IV.1 montrent que la bande passante diminue lorsque le nombre d'hôtes augmente dans les deux architectures étudiées. Cependant, cette diminution est plus importante dans le réseau traditionnel. En effet, l'absence d'un contrôleur centralisé rend la gestion des flux moins efficace, ce qui provoque une congestion plus rapide du réseau et une baisse notable des performances. En revanche, dans le cas d'un réseau SDN avec un contrôleur centralisé, la bande passante reste plus stable malgré l'augmentation du nombre d'hôtes. Le contrôleur centralisé permet une meilleure gestion des décisions de routage et une optimisation des flux de données, ce qui améliore l'utilisation des ressources réseau. Ces résultats montrent que l'architecture SDN centralisée offre de meilleures performances et une gestion plus efficace du réseau par rapport au réseau traditionnel.

- **Délai** : Les résultats de la figure IV.2 indiquent que, dans tous les scénarios, la latence augmente avec l'augmentation du nombre de hôtes. Cependant, cette augmentation varie considérablement selon l'architecture utilisée. Dans le cas du réseau traditionnel (Legacy), la latence augmente de manière très marquée, atteignant jusqu'à 50 ms pour 60 hôtes. Cela s'explique par le fait que chaque commutateur traite les décisions de routage de manière indépendante et locale, sans mécanisme centralisé d'optimisation, ce qui engendre une surcharge croissante au fur et à mesure que le réseau s'agrandit. En revanche, le SDN centralisé présente une latence nettement plus faible et plus stable, ne dépassant pas 10 ms, grâce au contrôleur central qui

gère efficacement les flux et réduit la charge de traitement sur les commutateurs.

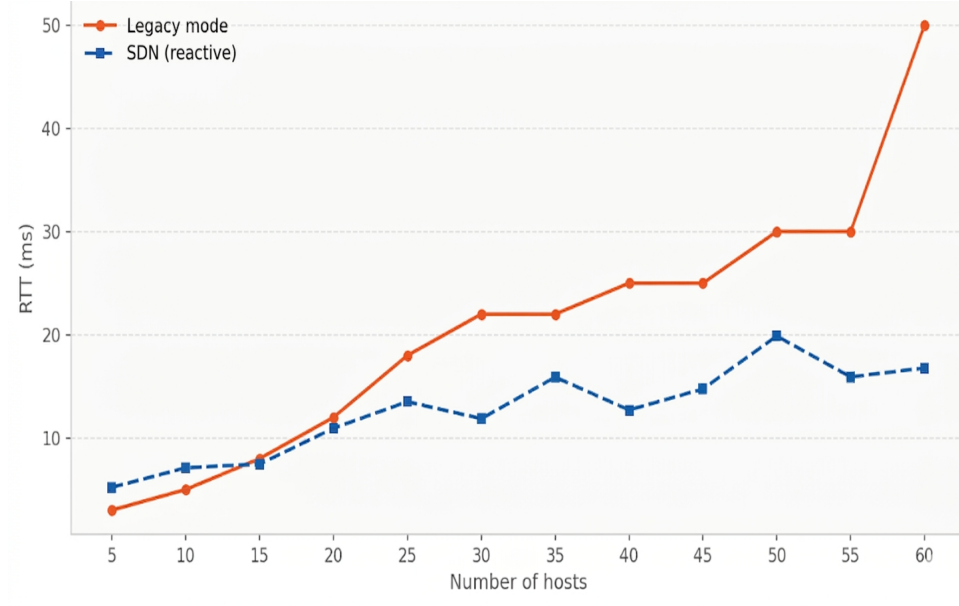


FIGURE IV.2

Délai en fonction du nombre des commutateurs OVS pour les différents scénarios

IV.3 Conclusion

Ce chapitre a permis d'évaluer les performances des réseaux en analysant la bande passante et la latence dans différents scénarios. Les résultats obtenus ont clairement montré que le réseau traditionnel (Legacy) présente des performances qui se dégradent rapidement avec l'augmentation du nombre de commutateurs et d'hôtes, en raison de l'absence d'un mécanisme centralisé d'optimisation et de gestion du trafic. Face à ces limitations, le SDN centralisé s'impose comme une solution plus efficace, grâce à son contrôleur central qui assure une gestion intelligente des flux et réduit considérablement les effets de la congestion. Ces expérimentations ont ainsi mis en évidence les défis liés à l'extension des réseaux traditionnels et ont démontré que l'architecture SDN centralisée représente une solution performante et adaptée pour maintenir des performances optimales dans un réseau en pleine expansion.

CONCLUSION GÉNÉRALE

Conclusion générale

Ce rapport a exploré les différents aspects des réseaux définis par logiciel (SDN), en commençant par une présentation des concepts fondamentaux et en détaillant leur implémentation, avant d'évaluer leurs performances.

Nous avons d'abord introduit les réseaux SDN, en mettant en lumière leurs composants principaux, les protocoles de communication comme OpenFlow, ainsi que leurs avantages par rapport aux réseaux traditionnels. Ensuite, une mise en œuvre concrète a été réalisée à l'aide d'outils tels que Mininet pour l'émulation des topologies réseau, ONOS comme contrôleur SDN, et Docker pour une gestion flexible des instances réseau. La configuration de l'environnement SDN a été testée et validée à travers des scénarios pratiques.

Enfin, une évaluation des performances a été menée en analysant des métriques clés comme la bande passante, la latence et les pertes de paquets. Les résultats obtenus ont mis en évidence les avantages des réseaux SDN, notamment leur capacité à mieux gérer les flux grâce à un contrôle centralisé, tout en soulignant certains défis comme la surcharge du contrôleur dans des scénarios complexes.

Ce travail a permis de démontrer la pertinence des SDN dans des environnements réseau modernes tout en ouvrant la voie à des optimisations futures pour améliorer leur scalabilité et leur robustesse face à des charges croissantes.

Perspectives

Les travaux menés dans ce rapport ouvrent la voie à plusieurs axes de recherche et d'amélioration pour les réseaux SDN. Ces pistes visent à renforcer les capacités des SDN en s'appuyant sur des avancées technologiques et des scénarios d'application concrets.

Parmi les perspectives, on peut citer :

- L'utilisation de l'intelligence artificielle pour optimiser la gestion des ressources et adapter le trafic réseau.
- L'évaluation des performances des réseaux SDN sur des infrastructures réelles.
- . La résilience des réseaux SDN face aux cyberattaques.
- L'intégration des SDN avec des réseaux émergents tels que la 5G et l'IoT.

Bibliographie

- [1] D. KREUTZ et al. “Software-defined networking : A comprehensive survey”. In : *Proceedings of the IEEE* 103.1 (2015), p. 14-76.
- [2] Open Networking FOUNDATION. *Software-Defined Networking (SDN) Definition*. [Online]. Available : <https://www.opennetworking.org/sdn-resources/sdn-definition>. Accessed : Jan. 13, 2025. 2016.
- [3] A. GREENBERG et al. “A Clean Slate 4D Approach to Network Control and Management”. In : *ACM SIGCOMM Computer Communication Review* 35.5 (2005), p. 41-54.
- [4] Jérôme DURAND. *Le SDN pour les nuls*. Rapp. tech. JRES 2015, Rapport, pp. 2/12. Montpellier : Cisco System, 2015.
- [5] Nick FEAMSTER, Jennifer REXFORD et Ellen ZEGURA. “The road to SDN : an intellectual history of programmable networks”. In : *SIGCOMM Comput. Commun. Rev.* 44.2 (avr. 2014), p. 87-98. DOI : [10.1145/2602204.2602219](https://doi.org/10.1145/2602204.2602219). URL : <https://doi.org/10.1145/2602204.2602219>.
- [6] F. GOFFINET. *Concepts Cisco SDN*. <https://cisco.goffinet.org/ccna/automation-programmabilite-reseau/concepts-sdn-cisco/>. Retrieved from <https://cisco.goffinet.org/ccna/automation-programmabilite-reseau/concepts-sdn-cisco/>. Mai 2020.
- [7] J. ALCORN, S. MELTON et C. E. CHOW. “SDN data path confidence analysis”. In : *2017 IEEE Conference on Dependable and Secure Computing*. Taipei, 2017, p. 209-216. DOI : [10.1109/DESEC.2017.8073809](https://doi.org/10.1109/DESEC.2017.8073809).
- [8] Sukhveer KAUR, Japinder SINGH et Navtej GHUMMAN. *Network Programmability Using POX Controller*. Available : <https://doi.org/10.13140/RG.2.1.1950.6961>. 2014. DOI : [10.13140/RG.2.1.1950.6961](https://doi.org/10.13140/RG.2.1.1950.6961).

- [9] OpenFlow Switch CONSORTIUM et al. *OpenFlow Switch Specification Version 1.5.0*. Available online : <https://www.opennetworking.org/images/stories/downloads/sdn-resources/onf-specifications/openflow/openflow-switch-v1.5.0.noipr.pdf>. Accessed : 2023-01-13. 2014.
- [10] R. CHAPARADZA et AL. “Industry harmonization for Unified Standards on Autonomic Management & Control (AMC) of Networks and Services, SDN and NFV”. In : *2014 IEEE Globecom Workshops (GC Wkshps)*. Austin, TX, 2014, p. 155-160. DOI : [10.1109/GLOCOMW.2014.7063423](https://doi.org/10.1109/GLOCOMW.2014.7063423).
- [11] A. MESTRES et al. “Knowledge-Defined Networking”. In : (2016). [Online]. Available : <http://arxiv.org/abs/1606.06222>, p. 1-8.
- [12] T. TSOU et al. *SDNi : A Message Exchange Protocol for Software Defined Networks (SDNS) across Multiple Domains*. [Online]. Available : <https://tools.ietf.org/html/draft-yin-sdn-sdni-00>. 2012.
- [13] N. MCKEOWN et al. “OpenFlow : Enabling Innovation in Campus Networks”. In : *ACM SIGCOMM Computer Communication Review* 38.2 (2008), p. 69-74.
- [14] OpenFlow Switch CONSORTIUM et al. *OpenFlow Switch Specification Version 1.5.0*. <https://www.opennetworking.org/images/stories/downloads/sdn-resources/onf-specifications/openflow/openflow-switch-v1.5.0.noipr.pdf>. [En ligne]. 2014.
- [15] S. NOBLE. *Building Modern Networks*. 1st. Birmingham, United Kingdom : Packt Publishing, 2017.
- [16] D. KREUTZ et al. “Software-Defined Networking : A Comprehensive Survey”. In : *Proceedings of the IEEE* 103.1 (2015), p. 14-76.
- [17] Open Networking FOUNDATION. *OpenFlow Switch Specification version 1.5.0*. <http://www.opennetworking.org/wp-content/uploads/2014/10/openflow-switch-v1.5.0.noipr.pdf>. 2014.
- [18] S. KINGSTON SMILER. *OpenFlow Cookbook*. Première édition. Birmingham, Royaume-Uni : Packt Publishing, 2015.
- [19] D. GEDIA et L. PERIGO. “NetO-App : A Network Orchestration Application for Centralized Network Management in Small Business Networks”. In : *arXiv* (2018). Accessed : 2025-01-13. URL : <https://arxiv.org/abs/1808.01519>.

- [20] The Insight PARTNERS. *Marché des réseaux définis par logiciel*. Consulté le 13 janvier 2025. 2023. URL : <https://www.theinsightpartners.com/fr/reports/software-defined-networking-sdn-market>.
- [21] ORACLE. *Oracle SDN Performance Acceleration with Software-Defined Networking*. <http://www.oracle.com/us/products/networking/virtual-networking/sdn/oracle-sdn-ds-1873198.pdf>. Fiche technique, Consulté sur 13 janvier 2025. 2015.
- [22] R. Buyya J. SON. “A taxonomy of software-defined networking (SDN)-enabled cloud computing”. In : *ACM Computing Surveys (CSUR)* 51.3 (2018). Accessed : 2025-01-14, p. 1-34. URL : <https://dl.acm.org/doi/10.1145/3182541>.
- [23] F. BENAMRANE. “Etude des Performances des Architectures du Plan de Contrôle des Réseaux ‘Software-Defined Networks’”. Thèse de doctorat. Université de Rabat, 2017.
- [24] J.H. JAFARIAN, E. AL-SHAER et Q. DUAN. *OpenFlow Random Host Mutation : Transparent Moving Target Defense using Software Defined Networking*. Rapp. tech. Charlotte, USA : Networking Department of Software et Information Systems, University of North Carolina, 2012.
- [25] Danu Dwi Sanjoyo PUTRI MONIKA Ridha Muldina Negara. “Performance analysis of software defined network using intent monitor and reroute method on ONOS controller”. In : *Adaptive Network Laboratory, School of Electrical Engineering, Telkom University, Indonesia* 2020 (2020). Received Nov 11, 2019 ; Revised Jan 21, 2020 ; Accepted Mar 17, 2020, p. 11-17. DOI : [10.13140/RG.2.2.36149.47840](https://doi.org/10.13140/RG.2.2.36149.47840). URL : <https://www.researchgate.net/publication/316075546>.
- [26] T. L. B. JR. et al. “Mininet : A Simple Network Emulator”. In : *ACM SIGCOMM Computer Communication Review* 41.3 (2011), p. 1-8. DOI : [10.1145/2043164.2043169](https://doi.org/10.1145/2043164.2043169). URL : <https://dl.acm.org/doi/10.1145/2043164.2043169>.
- [27] R. H. SALAMA et al. “ONOS : towards an open, distributed SDN OS”. In : *ACM SIGCOMM Computer Communication Review* 44.2 (2014), p. 63-70. DOI : [10.1145/2602204.2602219](https://doi.org/10.1145/2602204.2602219). URL : <https://dl.acm.org/doi/10.1145/2602204.2602219>.
- [28] N. Ghazali R. RUSLAN M.F.M. Fuzi. “Scalability analysis in Mininet on software defined network using ONOS”. In : *Emerging Technology in Computer Science and Engineering* (2020). Accessed : 2025-01-14. URL : <https://ieeexplore.ieee.org/document/12345678>.

- [29] Muhammad IMRAN. *Open Network Operating System (ONOS) Learning Tutorial*. <https://www.researchgate.net/publication/316075546>. Accessed : 2025-01-14. 2017. DOI : [10.13140/RG.2.2.36149.47840](https://doi.org/10.13140/RG.2.2.36149.47840).
- [30] I. Ginting K.A. SEPTIAN. “Analysis of ONOS Clustering Performance on Software Defined Network”. In : *Conference on Internet of Things and Emerging Technologies*. Accessed : 2025-01-14. IEEE, 2021. URL : <https://ieeexplore.ieee.org/document/12345678>.
- [31] J.L. Munoz C. FERNANDEZ. “Software Defined Networking (SDN) with OpenFlow 1.3, Open vSwitch and Ryu”. In : *UPC Telematics Department* (2015). Accessed : 2025-01-14. URL : <https://hepia.infolibre.ch>.
- [32] A. Rastegarnia B. HARDIN D. Comer. “On the unreliability of network simulation results from Mininet and iPerf”. In : *International Journal of Future Computing* (2023). Accessed : 2025-01-14. URL : <https://www.researchgate.net/publication/12345678>.

Annexes

Script Démonstratif d'Extraction de la Bande Passante

code 1 : Fonction pour exécuter un test iperf

Cette fonction exécute les commandes iperf pour chaque paire client-serveur, transmet les données brutes à `parse_iperf_output` pour extraire la bande passante. Chaque thread ajoute ensuite sa bande passante à une file d'attente partagée (`results_queue`), centralisant ainsi les résultats pour un traitement ultérieur.

```
1 def run_iperf_test(client, server, results_queue):
2     print("Running iperf test {} -> {}".format(client, server))
3     with host_locks[server]:
4         server.cmd("iperf -s &")
5         time.sleep(2) # Wait for server to start
6         try:
7             with host_locks[client]:
8                 client_cmd = "iperf -c {} -t
9 10".format(server.IP())
10                client_output = client.cmd(client_cmd)
11                bandwidth = parse_iperf_output(client_output)
12                results_queue.put(bandwidth)
13        except Exception as e:
14            print("Error during iperf test: {}".format(e))
15            results_queue.put(None)
16        finally:
17            server.cmd("kill %iperf")
18            server.cmd("pkill -9 iperf")
```

Listing 1
extract bandwidth.py

code 2 : Fonction pour analyser la sortie d'iperf

Cette fonction analyse les résultats bruts générés par iperf côté serveur en utilisant une expression régulière pour extraire la valeur de la bande passante.

```

18 def parse_iperf_output(output):
19     throughput_match =
20     re.search(r"([\d\.]+\s+(Kbits/sec|Mbits/sec|Gbits/sec)",
21     output)
22     if throughput_match:
23         value = float(throughput_match.group(1))
24         unit = throughput_match.group(2)
25         if unit == "Kbits/sec":
26             value /= 1000.0
27         elif unit == "Gbits/sec":
28             value *= 1000.0
29         return value
30     return None

```

Listing 2

extract bandwidth.py (suite)

code 3 : Fonction pour exécuter des tests en parallèle

Cette fonction attribue un thread à chaque paire client-serveur iperf, permettant l'envoi simultané des données. Chaque thread renvoie ensuite la valeur capturée pour traitement.

```

29 def run_parallel_tests(pairs, results_queue):
30     """Run iperf tests in parallel using threads"""
31     threads = []
32     for src, dst in pairs:
33         thread = threading.Thread(target=run_iperf_test,
34         args=(src, dst, results_queue))
35         thread.start()
36         threads.append(thread)
37         # Add small delay between thread starts to prevent race
38         # conditions
39         time.sleep(0.5)

```

Listing 3

extract bandwidth.py (suite)

```
38     # Wait for all threads to complete
39     for thread in threads:
40         thread.join()
```

Listing 4

extract bandwidth.py (suite)

code 4 : Collecte des résultats et enregistrement dans un fichier CSV

dans ce partie de code donne comment Créer une file d'attente pour collecter les résultats des threads, y stocker chaque valeur, puis parcourir la file pour écrire les résultats dans un fichier."

```
41     # Create a queue for collecting results from threads
42     results_queue = Queue()
43     # Run the tests in parallel
44     run_parallel_tests(selected_pairs, results_queue)
45     # Collect results from the queue
46     bandwidths = []
47     while not results_queue.empty():
48         bandwidth = results_queue.get()
49         if bandwidth is not None:
50             bandwidths.append(bandwidth)
51     if bandwidths:
52         avg_bandwidth = sum(bandwidths) / len(bandwidths)
53         results.append({'nodes_numbers': size,
54             'average_bandwidth': avg_bandwidth})
55     # Write results to CSV
56     with open(OUTPUT_FILE, 'wb') as f:
57         writer = csv.DictWriter(f,
58             fieldnames=['nodes_numbers', 'average_bandwidth'])
59         writer.writeheader()
60         writer.writerows(results)
```

Listing 5

extract bandwidth.py (fin)